Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
int countZeroes(int a[], int low, int high)
{
    if (low > high)
        return 0;
    if (a[low] == 0)
        return high - low + 1;
    int mid = (low + high) / 2;
    return countZeroes(a, mid + 1, high);
}
int main()
{
    int m;
    scanf("%d", &m);
    int a[m];
    for (int i = 0; i < m; i++)
        scanf("%d", &a[i]);
    printf("%d", countZeroes(a, 0, m - 1));
    return 0;
}
```

	Input	Expected	Got	
✔	5 1 1 1 0 0	2	2	✔
✔	10 1 1 1 1 1 1 1 1 1 1	0	0	✔
✔	8 0 0 0 0 0 0 0 0	8	8	✔
✔	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
int count(int a[], int low, int high, int x)
{
    int c = 0;
    for (int i = low; i <= high; i++)
    {
        if (a[i] == x)
            c++;
    }
    return c;
}
int majority(int a[], int low, int high)
{
    if (low == high)
        return a[low];
    int mid = (low + high) / 2;
    int left = majority(a, low, mid);
    int right = majority(a, mid + 1, high);
    if (left == right)
        return left;
    int leftCount = count(a, low, high, left);
    int rightCount = count(a, low, high, right);
    if (leftCount > rightCount)
        return left;
    else
        return right;
}
int main()
{
    int n;
    scanf("%d", &n);
    int a[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    printf("%d", majority(a, 0, n - 1));
    return 0;
}
```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
int floorValue(int a[], int low, int high, int x, int floor)
{
    if (low > high)
        return floor;
    int mid = (low + high) / 2;
    if (a[mid] == x)
        return a[mid];
    if (a[mid] < x)
        return floorValue(a, mid + 1, high, x, a[mid]);
    return floorValue(a, low, mid - 1, x, floor);
}
int main()
{
    int n, x;
    scanf("%d", &n);
    int a[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    scanf("%d", &x);
    printf("%d", floorValue(a, 0, n - 1, x, -1));
    return 0;
}
```

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔
✔	5 10 22 85 108 129 100	85	85	✔
✔	7 3 5 7 9 11 13 15 10	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as “No”.

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value “x”)

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
void f(int a[], int l, int h, int x)
{
    if (l >= h)
    {
        printf("No");
        return;
    }
    int s = a[l] + a[h];
    if (s == x)
    {
        printf("%d\n%d", a[l], a[h]);
        return;
    }
    if (s < x)
        f(a, l + 1, h, x);
    else
        f(a, l, h - 1, x);
}
int main()
{
    int n, x;
    scanf("%d", &n);
    int a[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    scanf("%d", &x);
    f(a, 0, n - 1, x);
    return 0;
}
```

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```c
#include <stdio.h>

int max(int a, int b)
{
    return (a > b) ? a : b;
}
int main()
{
    int n;
    scanf("%d", &n);
    int board[n][n];
    int dp[n][n];
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &board[i][j]);
        }
    }
    dp[0][0] = board[0][0];

    for (int j = 1; j < n; j++)
    {
        dp[0][j] = dp[0][j - 1] + board[0][j];
    }
    for (int i = 1; i < n; i++)
    {
        dp[i][0] = dp[i - 1][0] + board[i][0];
    }
    for (int i = 1; i < n; i++)
    {
        for (int j = 1; j < n; j++)
        {
            dp[i][j] = board[i][j] +max(dp[i - 1][j], dp[i][j - 1]);
        }
    }
    printf("%d\n", dp[n - 1][n - 1]);

    return 0;
}

```

	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.